

MDTS Web Dashboard + AI Backend

TL;DR

`frontend_aibackend` 는 선박용 엣지 AI 의료 지원 시스템인 MDTS(Maritime Digital Triage System)의 웹 대시보드와 AI 백엔드를 결합한 통합 모듈이다. React 기반 웹 화면, Node.js API 브리지, FastAPI AI 서버, Ollama LLM, ChromaDB RAG, MariaDB 바이탈/선원 데이터, PyQt5 장비 연동 API가 하나의 흐름으로 동작한다.

이 모듈의 목표는 선박 안에서 의사가 없는 상황에서도 선원 상태 조회, 실시간 바이탈 확인, 외상 촬영 결과 연동, 응급처치 세션 기록, 환자 차트 저장, AI 질의응답, 상황 대응 타임라인을 하나의 대시보드에서 운영하는 것이다.

프로젝트 목적

MDTS는 해상 환경에서 통신 지연, 의료진 부재, 장비 제약이 동시에 발생한다는 전제를 둔다. 따라서 시스템은 온라인 DB와 엣지 장비를 함께 사용하되, 현장 시연과 실제 운용 모두에서 다음 기준을 만족해야 한다.

- 실시간 센서값을 웹 대시보드와 PyQt5 화면에 동일하게 반영한다.
- 선원별 의료 정보, 혈액형, 기저질환, 알레르기, 과거 기록을 DB에서 불러온다.
- 웹에서 AI에게 질문하면 MariaDB의 최신 바이탈과 선원 정보를 우선 조회한다.
- AI 답변은 ChromaDB RAG와 Obsidian Markdown 지식 근거를 참고하되, 화면에는 원문/파일명/테이블 정의를 노출하지 않는다.
- 외상 촬영은 노트북 카메라가 아니라 Jetson Nano PyQt5 카메라 흐름을 기준으로 동작한다.
- 응급처치 세션 종료 보고는 환자 차트의 지난 기록으로 저장된다.
- 집중 관리 중인 선원 상태는 웹과 PyQt5가 동일하게 유지한다.

전체 아키텍처

```
[React Web Dashboard]
  |
  | HTTP :4000
  ▼
[Node.js Dashboard API]
  ├── Campus MySQL / MariaDB : tb_crew, tb_vital, tb_analysis, tb_firstaid
  ├── Raspberry Pi Sensor API :5000
  └── FastAPI AI Backend :8000
      |
      ├── Ollama llama3.2:1b / nomic-embed-text
      ├── ChromaDB Vector DB
      └── Obsidian Markdown knowledge source

[Raspberry Pi Sensor Server]
  ├── MAX30102 / MLX90614 / ADS1115 sensor collection
  └── local MariaDB vital_logs
```

- └─ focused crew shared state
- └─ Jetson PyQt5 control proxy

[Jetson Nano PyQt5]

- └─ crew selection and focused patient registration
- └─ real-time vital display
- └─ trauma camera capture
- └─ first-aid guide screen

디렉터리 구조

```
frontend_aibackend/  
└─ src/  
  └─ App.jsx  
  └─ pages/  
    └─ Main.jsx  
    └─ CrewManagement.jsx  
    └─ Emergency.jsx  
    └─ PatientChart.jsx  
  └─ pages/Main/components/  
    └─ DashboardView.jsx  
    └─ MainTutorial.jsx  
  └─ utils/  
    └─ api.js  
    └─ avatar.js  
    └─ AlertContext.jsx  
  └─ components/  
└─ server/  
  └─ index.js  
  └─ package.json  
└─ ai_backend/  
  └─ m_medic_server.py  
  └─ m_medic_knowledge_engine.py  
  └─ maritime_medical_knowledge.txt  
  └─ medical_vector_db/  
  └─ M_MEDIC_v2/04_integrated_system/  
└─ tools/  
  └─ jetson_ollama_tunnel.py  
  └─ start_remote_trauma_stack.py  
└─ public/assets/  
└─ package.json  
└─ start_full_stack.mjs  
└─ start_ai_backend.mjs
```

핵심 런타임 구성

구성 요소	기본 포트	역할
Vite React Frontend	5174	웹 대시보드 UI
Node Dashboard API	4000	DB, 센서, PyQt5, 파일 export, 환자 기록 API
FastAPI AI Backend	8000	AI 질의응답, RAG, 바이탈/선원 조회
Ollama	11434	LLM 및 임베딩 모델 실행
Raspberry Pi Sensor API	5000	센서값, 집중 관리 상태, Jetson 제어 프록시
Jetson PyQt5 Control API	5055	외상 촬영 시작/중지/결과/가이드 전환

주요 화면과 기능

1. Login

로그인 페이지는 시리얼 번호, 기기 번호, 선박 번호를 입력받아 웹 세션에 저장한다. 시연 환경에서는 이 값이 사용자 접속 및 장비 식별 로그로 활용된다.

2. Main Dashboard

메인 페이지는 현재 선택된 선원의 정보를 기준으로 다음 항목을 표시한다.

- 실시간 심박수, 산소포화도, 호흡수, 혈압, 체온
- 선원 기본 정보와 의료 메타데이터
- AI 질의응답 채팅
- 외상 촬영 및 AI 분석 진입
- 상황 대응 타임라인
- 집중 관리 중인 선원 목록

메인 페이지의 선원 목록은 전체 선원이 아니라 **집중 관리 중** 상태인 선원만 표시한다. 이 상태는 웹 선원 관리와 PyQt5 환자 등록 양쪽에서 변경할 수 있으며 Raspberry Pi 센서 서버의 공유 상태를 통해 동기화된다.

3. Crew Management

선원 관리 페이지는 선원 조회, 검색, 부서별 필터, 신규 선원 등록, 선원 정보 수정, 환자 전환을 담당한다.

- **+ 환자로 전환** 을 누르면 해당 선원이 집중 관리 대상으로 등록된다.
- 집중 관리 대상은 버튼이 **집중 관리 중** 상태로 바뀐다.
- 이 상태는 **localStorage** 에 저장되고 동시에 Raspberry Pi의 집중 관리 상태 API로 동기화된다.
- PyQt5에서 환자 등록을 해도 웹 선원 관리 상태가 동일하게 반영된다.

4. Patient Chart

환자 차트는 현재 선원의 상세 프로필, 실시간 센서 데이터, 상태 작성 기록, 지난 기록 보기를 제공한다.

- 응급처치 세션 종료 보고가 **tb_patient_history** 에 저장되면 지난 기록 보기에서 확인된다.

- 지난 기록과 환자 정보를 Excel 형식으로 내보낼 수 있다.
- 상태 작성 저장 후에는 메인 페이지로 이동하고 외상 촬영 영역을 강조한다.

5. Trauma Capture

웹에서 외상 촬영을 누르면 로컬 노트북 카메라를 쓰지 않고 Jetson Nano PyQt5 카메라를 사용한다.

흐름은 다음과 같다.

```
Web Main trauma button
→ Node API /api/trauma/pyqt5/start
→ Pi Sensor API /trauma/start
→ Jetson PyQt5 /trauma/start
→ PyQt5 trauma camera screen
→ Web polls /api/trauma/pyqt5/frame.jpg and /result
```

촬영 완료 후 AI 분석 시작을 누르면 웹은 응급처치 페이지로 이동하고 PyQt5도 동일 외상 유형의 응급처치 가이드 화면으로 전환한다.

6. Emergency

응급처치 페이지는 외상 분석 결과 또는 사용자가 선택한 응급처치 항목을 기준으로 단계별 가이드를 제공한다.

- 우측 바이탈 패널은 PyQt5 센서값과 동일하게 표시된다.
- 임계치를 넘은 바이탈만 경고 애니메이션을 표시한다.
- 데이터 전송은 응급처치 세션 종료 보고를 환자 차트 지난 기록에 저장한다.
- 처치 종료는 데이터 전송 없이 메인 페이지로 이동한다.
- 데이터 전송 또는 처치 종료 시 PyQt5 카메라 송출/촬영 신호를 중단한다.
- 데이터 전송 후 메인 복귀 시 상황 대응 타임라인에 처리 결과를 남긴다.

AI 질의응답 설계

AI 응답은 다음 우선순위로 구성된다.

1. MariaDB의 최신 선원 정보와 바이탈 정보
2. 질문에 포함된 명시 요청
3. ChromaDB RAG 검색 결과
4. Obsidian Markdown 지식 근거
5. 일반 응급처치 원칙

질문이 혈액형, 현재 혈압 처럼 DB 사실 조회인 경우에는 LLM 추론을 거치지 않고 DB 최신값을 직접 반환한다. 예를 들어 한통신의 현재 혈압하고 혈액형 알려줘 라는 질문은 `tb_crew.bloodtype` 과 `tb_vital.blood_pressure` 를 조회해 답변한다.

질문이 바이탈 이상 여부 분석인 경우에는 규칙 기반 판정과 AI 답변을 분리한다. 0으로 들어온 심박수, 산소포화도, 호흡수, 체온은 사망/위험 수치가 아니라 미측정 또는 미전송으로 해석한다.

RAG와 Obsidian Markdown 원칙

AI 백엔드는 `m_medic_knowledge_engine.py` 를 통해 ChromaDB를 검색하고, Obsidian Markdown 문서를 보조 지식으로 검색한다.

운영 원칙은 다음과 같다.

- RAG와 Obsidian MCP/Markdown 근거는 내부 판단에만 사용한다.
- 화면 답변에는 `source`, `score`, `distance`, 파일명, DB 컬럼 정의, Markdown 표를 노출하지 않는다.
- 응급처치 가이드 질문은 실제 의료 대응 문장만 출력한다.
- 개발 문서, 버그 메모, UI 파일명 등은 사용자 의료 답변에 섞이지 않도록 필터링한다.

Node API 주요 엔드포인트

Method	Endpoint	역할
GET	<code>/api/crew</code>	원격 DB 선원 목록 조회
GET	<code>/api/vital/latest/:crewId</code>	선원별 최신 바이탈 조회
POST	<code>/api/vital</code>	바이탈 수기/웹 저장
GET	<code>/api/sensor/live</code>	Raspberry Pi 실시간 센서값 조회
GET	<code>/api/sensor/crew</code>	PyQt5 현재 선택 선원 조회
POST	<code>/api/sensor/crew</code>	현재 모니터링 선원 설정
GET	<code>/api/sensor/crew/focus</code>	집중 관리 선원 목록 조회
POST	<code>/api/sensor/crew/focus</code>	집중 관리 선원 목록 저장
POST	<code>/api/trauma/pyqt5/start</code>	PyQt5 외상 촬영 화면 시작
POST	<code>/api/trauma/pyqt5/capture</code>	PyQt5 촬영/분석 시작
GET	<code>/api/trauma/pyqt5/frame.jpg</code>	PyQt5 카메라 프레임 프록시
GET	<code>/api/trauma/pyqt5/result</code>	PyQt5 분석 결과 조회
POST	<code>/api/trauma/pyqt5/guide</code>	PyQt5 응급처치 가이드 전환
POST	<code>/api/trauma/pyqt5/stop</code>	PyQt5 카메라 송출 중단
GET	<code>/api/patient-history/:crewId</code>	환자 지난 기록 조회
POST	<code>/api/patient-history</code>	환자 기록 저장
GET	<code>/api/db/status</code>	DB 연결 상태 확인

FastAPI AI 주요 엔드포인트

Method	Endpoint	역할
GET	/health	AI 서버, DB, 모델 상태 확인
GET	/vitals/live?crew_id=<id>	최신 바이탈 조회
POST	/analyze/chat	AI 질의응답
POST	/analyze/wound	외상 이미지 분석 API
POST	/rag/reload	Markdown/RAG 지식 재적재

실행 방법

사전 조건

- Node.js 20 이상 권장
- Python 3.10 이상 권장
- Ollama 설치 및 모델 준비
- MariaDB/MySQL 접근 가능
- Raspberry Pi 센서 서버 접근 가능
- Jetson Nano PyQt5 제어 API 접근 가능

전체 실행

```
cd frontend_aibackend
npm install
npm run dev:all
```

개별 실행

```
# React frontend
npm run dev:frontend

# Node dashboard API
npm run dev:api

# FastAPI AI backend
npm run dev:ai
```

상태 확인

```
curl http://localhost:4000/api/db/status
curl http://localhost:8000/health
curl http://localhost:11434/api/tags
```

환경 변수

변수	목적
MDTS_FRONTEND_PORT	Vite 프론트엔드 포트 변경
MDTS_AI_PYTHON	AI 백엔드 실행 Python 지정
MDTS_OLLAMA_HOST	Ollama 호스트 지정
MDTS_OLLAMA_PORT	Ollama 포트 지정
MDTS_OBSIDIAN_DIR	Obsidian Markdown 원본 경로
MDTS_OBSIDIAN_AUTO_SYNC	Markdown 자동 동기화 활성화
MDTS_OBSIDIAN_SYNC_SEC	자동 동기화 주기

데이터 저장 기준

데이터	저장 위치	비고
선원 기본 정보	원격 <code>tb_crew</code>	웹/AI 공통 참조
실시간 바이탈	원격 <code>tb_vital</code>	PyQt5 선택 선원 기준 저장
로컬 센서 로그	Pi <code>vital_logs</code>	USB CSV 백업 병행
환자 지난 기록	Pi <code>tb_patient_history</code>	응급처치 세션 기록 포함
집중 관리 상태	Pi JSON state	웹/PyQt5 공유 상태
RAG 지식	ChromaDB	Vector DB + Markdown 기반

집중 관리 상태 동기화

집중 관리 상태는 원격 DB 테이블을 수정하지 않고 Raspberry Pi 센서 서버의 공유 상태 API로 동기화한다.

```
Web CrewManagement
→ POST /api/sensor/crew/focus
→ Pi /crew/focus
→ /home/pi/mdts_focused_crew_state.json

PyQt5 CrewScreen
→ POST Pi /crew/focus
→ Web App polling
→ localStorage mdts_crew_list 갱신
```

이 구조는 시연용으로 빠르고 안전하다. 기존 ERD나 원격 DB 테이블에 영향을 주지 않는다.

상황 대응 타임라인 기준

타임라인은 AI가 임의 판단해 쓰지 않는다. 다음 실제 이벤트만 기록한다.

- 메인 선원 조회
- 실시간 바이탈 최초 수신
- 바이탈 큰 폭 변화
- AI 질문 요청
- AI 답변 완료
- 응급처치 데이터 전송 완료

응급처치 종료 보고는 메인 복귀 시 대상자, 처치 항목, 진단 기준, 완료 단계, 세션 시간, 저장 결과, 마지막 바이탈 요약으로 표시된다.

트러블슈팅

증상	원인	조치
AI 응답이 너무 빠르고 내용이 단순함	LLM 호출 전 규칙 기반 분기	질문 의도 분류와 <code>/health</code> 의 <code>model_loaded</code> 확인
<code>HTTPConnectionPool 11434</code> 오류	Ollama 미실행 또는 호스트 불일치	Ollama 실행, tunnel 또는 <code>MDTS_OLLAMA_HOST</code> 확인
혈액형이 AI 답변에 누락됨	DB 사실 조회가 바이탈 분석으로 분류됨	<code>patient_fact_lookup</code> 분기 확인
메인에서 카메라가 자동 재오픈 됨	PyQt5 마지막 촬영 상태가 재감지 됨	<code>/api/trauma/pyqt5/stop</code> 호출 및 이벤트 ID 무시 상태 확인
선원 목록에 전체 선원이 보임	집중 관리 필터 미적용	<code>DashboardView.jsx</code> 의 <code>isEmergency</code> 필터 확인
PyQt5와 웹 집중 관리 상태가 다름	Pi <code>/crew/focus</code> 동기화 실패	Node API와 Pi Sensor API 상태 확인
환자 차트 지난 기록이 비어 있음	<code>tb_patient_history</code> 저장 실패	Node API 로그와 Pi MariaDB 연결 확인
체온 센서를 켜는데 값이 남음	이전 수기값/센서값 캐시	<code>/recording</code> , <code>/manual/clear</code> 동작 확인

보안 및 운영 주의

이 저장소는 데모 환경을 포함하므로 private 저장소로 유지한다. README에는 비밀번호를 직접 기록하지 않는다. 실제 운영 전에는 다음을 진행한다.

- DB, SSH, API 접속정보를 환경 변수 또는 secret store로 분리
- 원격 DB 계정 권한 최소화
- 환자/선원 개인정보 마스킹 정책 적용

- API 인증 토큰 또는 내부망 접근 제한 추가
- 로그 파일에 PII와 의료정보가 과다 저장되지 않도록 필터링

개발 원칙

- 원본 폴더는 보존하고 `04_frontend_aibackend` 결합본에서만 개발한다.
- 웹과 PyQt5 상태가 갈라지는 기능은 반드시 Pi 공유 API 또는 DB를 기준으로 동기화한다.
- AI 답변 품질 문제는 프롬프트만 수정하지 말고 DB 조회, RAG 검색, 후처리 필터를 함께 확인한다.
- 의료 답변에는 개발 문서, Markdown 원문, DB 스키마 표가 섞이면 안 된다.
- 시연 중 장비 메모리가 부족하면 Ollama keep_alive, 백그라운드 프로세스, PyQt5 카메라 타이머를 우선 점검한다.

릴리즈 체크리스트

- React 웹 대시보드 실행 확인
- Node API `4000` 실행 확인
- FastAPI AI `8000` 실행 확인
- Ollama 모델 목록 확인
- Pi Sensor API `5000` 확인
- PyQt5 화면 실행 확인
- 집중 관리 상태 웹/PyQt5 양방향 확인
- PyQt5 외상 촬영 시작/중지 확인
- 응급처치 데이터 전송 후 환자 차트 기록 확인
- AI 질문 시 DB 사실 조회와 RAG 답변 분리 확인

문서 기준일

- 문서 기준일: 2026-05-13
- 대상 저장소: `Capernaum-user/mdts-maritime-medic-integrated-demo`
- 대상 모듈: `frontend_aibackend`